Software Requirements Specifications

Voice Coder AI

Project Code:

VCAI560

Internal Advisor:

Mr. Mudassir Ali Zaidi

Project Manager:

Professor Dr. Muhammad Ilyas

Project Team:

Alina Ahmed	**BSCS51F22S044**	**Team Lead**
Laiba Humayun	**BSCS51F22S031**	**Team Member**
Maryum Javed	**BSCS51F22S009**	**Team Member**

Submission Date:

May 10,2026

Project Supervisor's Signature

Document Information

Category	Information
Customer	University of Sargodha
Project	Voice Coder AI
Document	Requirement Specifications
Document Version	1.0
Identifier	VCAI560
Status	Draft
Author(s)	Alina Ahmed Laiba Humayun Maryum Javed
Approver(s)	Professor Dr. Muhammad Ilyas
Issue Date	October 20, 2025
Document Location	
Distribution	1. Professor Mudassir Ali Zaidi 2. Professor Dr. Muhammad Ilyas

Definition of Terms, Acronyms and Abbreviations

Term/Acronym	Description
SRS	Software Requirements Specification
IEEE	Institute of Electrical and Electronics Engineers
C++	C Plus Plus
AI	Artificial Intelligence
API	Application Programming Interface
GPT-4o	Generative Pre-trained Transformer 4 Omni
JWT	JSON Web Token
SQL	Structured Query Language
HTTPS	HyperText Transfer Protocol Secure
TLS	Transport Layer Security
OWASP	Open Web Application Security Project
UI	User Interface
UX	User Experience
IDE	Integrated Development Environment
RAM	Random Access Memory
CLI	Command Line Interface
JSON	JavaScript Object Notation

Table of Contents

1. Introduction

1.1 Purpose of Document

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for the VoiceCoder AI web application. It is structured according to the IEEE Std 830-1998 [5] recommended practice. It serves as a guide for stakeholders, developers, and evaluators, defining the project's functionality, limitations, and design goals.

1.2Project Overview

VoiceCoder AI is a browser-based application that enables users to write C++ programs using voice commands. The system translates spoken instructions into syntactically correct C++ code using a three-level voice command parser, generates intelligent code through GPT-4o AI integration [12], compiles it in real time via cloud compiler APIs, and displays the output. The objective is to improve accessibility for users with physical disabilities or RSI, promote hands-free programming, and provide AI-assisted coding for Pakistani students including Urdu Text-to-Speech support.

1.3Scope

The system will:

- *Convert voice commands into valid C++ code using the Web Speech API [1].*
- *Support basic to intermediate C++ constructs through predefined voice command mappings.*
- *Generate complete C++ code from natural language voice requests using GPT-4o AI [12].*
- *Provide real-time cloud-based compilation via Judge0, Paiza.io, and JDoodle APIs [3], [4].*
- *Display program outputs and errors in the output console.*
- *Allow users to save, load, delete, and share code files through cloud storage [13].*
- *Authenticate users securely using JWT tokens and bcrypt password hashing [7].*
- *Provide Urdu Text-to-Speech so users can hear code and AI responses in Urdu [1].*
- *Operate on modern browsers without installation.*

The system will not:

- *Support programming languages other than C++.*
- *Offer advanced IDE features such as version control or integrated debugging.*

- *Work fully offline as cloud APIs are required.*
- *Handle complex C++ metaprogramming.*
- *Support iOS mobile devices.*

2. Overall System Description

It describes the environment, in which the system will be developed and used, the anticipated users of the system and the known constraints, assumptions and dependencies.

2.1User characteristics

Primary Users: *Students with disabilities, programmers with RSI, computer science students, and Pakistani developers.*

Secondary Users: *Accessibility researchers and AI-assisted coding users.*

2.2Operating environment

- *Hardware: PC, laptop, tablet with microphone.*
- *Operating System: Windows, macOS, Linux.*
- *Browser: Chrome 70+ or Edge 79+ [1].*
- *Backend: Node.js server on Railway [14].*
- *Database: PostgreSQL on Supabase [13].*
- *Dependencies: Web Speech API [1], Monaco Editor [2], GPT-4o API [12], Paiza.io / JDoodle / Judge0 APIs [3], [4], JWT, bcryptjs.*

2.3System constraints

- *Requires internet connection for AI, voice, and compilation.*
- *Limited to C++ language features supported by compiler APIs [3], [4].*
- *Needs microphone and clear speech input.*

- *GPT-4o features require API key [12].*

3. External Interface Requirements

This section is intended to specify any requirements that ensure that our system will connect properly to external components.

3.1Hardware Interfaces

The system is fully web-based and only requires a microphone and internet connection.

3.2Software Interfaces

- *Frontend: HTML, CSS, JavaScript.*
- *Code Editor: Monaco Editor [2].*
- *Speech Recognition: Web Speech API [1].*
- *AI Engine: GPT-4o API [12].*
- *Compilation: Judge0, Paiza.io, JDoodle APIs [3], [4].*
- *Backend: Node.js + Express.*
- *Database: Supabase PostgreSQL [13].*
- *Authentication: JWT + bcryptjs [7].*
- Hosting: Railway cloud [14].

3.3Communications Interfaces

- *All communication uses HTTPS/TLS encryption.*
- *JWT-based authentication used for secure API access.*
- *Backend communicates with Supabase using secure credentials.*
- *Compiler API keys stored securely on server [7].*

4. Functional Requirements

4.1Voice Command Recognition

Uses Web Speech API for continuous voice input [1].
Noise filtering applied using Web Audio API.

4.2 Command Parsing and Code Generation

Level 1: Direct C++ snippets (loops, classes, print) [5]
Level 2: System commands (compile, save, undo)
Level 3: Natural language → GPT-4o code generation [12]

4.3 Code Display and Editing

Uses Monaco Editor [2] with syntax highlighting, tabs, autocomplete, and editing support.

4.4 Code Compilation and Execution

Backend sends code to Judge0, Paiza.io, or JDoodle [3], [4] and returns output.

4.5 AI Features

- *GPT-4o supports:*
- *Code generation*
- *Error fixing*
- *Code explanation*
- *Code review [12]*

4.6 Cloud File Management

Users can save/load/share code files using Supabase database [13].

4.7 User Authentication

JWT authentication with bcrypt hashing [7].

4.8 Urdu Text-to-Speech

Uses Speech Synthesis API [1] for Urdu output.

4.9 Error Handling

Handles voice errors, API failures, compilation errors, and AI errors.

4.10 User Interaction

Push-to-talk, voice commands, compiler controls, file management UI.
de review [12]

5. Non-functional Requirements

5.1Performance Requirements

Voice processing: 2–4 seconds.
Compilation: 5 seconds average.
AI response: 3–6 seconds [12].

5.2 Safety Requirements

Code runs in secure sandbox environments (Judge0, JDoodle, Paiza.io) [3], [4].

5.3 Security Requirements

- *OWASP compliance [7].*
- *Passwords hashed using bcrypt.*
- *JWT session security.*
- *No API keys exposed in frontend.*

5.4User Documentation

User manual and system documentation maintained using agile approach [6].

6. Assumptions and Dependencies

- *Requires stable internet.*
- *Depends on Web Speech API [1], GPT-4o [12], compiler APIs [3], [4], Supabase [13].*
- *Browser must support speech recognition.*
- *User provides OpenAI API key.*

7. References

[1] MDN, Web Speech API
https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

[2] Microsoft, Monaco Editor
https://microsoft.github.io/monaco-editor

[3] Paiza.io
https://paiza.io/en

[4] JDoodle Compiler API
https://www.jdoodle.com/compiler-api

[5] IEEE Std 830-1998

[6] Pressman, Software Engineering

[7] OWASP Top Ten
https://owasp.org/www-project-top-ten/

[8] W3C WCAG
https://www.w3.org/TR/WCAG22/

[9] cppreference
https://cppreference.com

[10] OpenAI GPT-4o
https://platform.openai.com/docs/models/gpt-4o

[11] Supabase Docs
https://supabase.com/docs

[12] Railway Hosting
https://docs.railway.app